

Python coding reference

A classroom-friendly language surface for flight, vision, decisions, loops, and safe missions.

NO INSTALL

AUTOMATIC WAITING

LOCAL TRANSLATION

SIM + REAL

FIRST-FLIGHT.PY

```
1 # First flight
2 take_off()
3 fly("forward", 1, 15)
4 land()
```

Write Python; Hopper runs the proven flight engine

The Python tab translates a focused classroom subset locally. Bluetooth, Wi-Fi vision, the simulator, Stop & Land, and the camera gallery still use the same app runtime.

SAFE-FIRST-MISSION.PY

```
1  # Commands wait automatically.
2  take_off()
3
4  try:
5      fly("forward", 1, 15)
6      hover()
7      take_photo()
8  finally:
9      land()
```

WHAT THE APP HANDLES

Promise-returning drone and vision calls receive automatic waiting. Indentation becomes nested program structure. A syntax problem is reported with its Python line number before the motors start.

WHAT STUDENTS WRITE

Use four spaces for each level. End if, elif, else, while, for, def, try, except, and finally headers with a colon. Press Tab or Enter after a colon and the editor inserts the indentation.

IMPORTANT

Do not write await, import modules, or paste ordinary desktop Python libraries. This is Hopper Python: a deliberately small surface that runs inside the browser or desktop app.

Flight lifecycle and timed motion

Seconds, degrees, directions, and power percentages are explicit so missions are easy to read and review.

take_off()

No args. Auto-awaits; returns None. Low battery (<=10%) or Bluetooth failure raises; a stopped run returns early.

land()

No args. Auto-awaits; returns None after the landing wait. Bluetooth/transport failure raises.

hover()

No args. Auto-awaits; zeros every motion axis, waits about 1 s, and returns None.

wait(seconds)

seconds: number >=0; required. Auto-awaits and returns None; negative/non-number becomes 0; Stop ends the wait early.

fly(direction, seconds=1, power=15)

direction: six strings; seconds >=0; power clamps -100..100. Auto-awaits, returns None; transport failure raises.

rotate(degrees=0, direction="clockwise")

degrees: number >=0; direction: "clockwise"/"counterclockwise". Auto-awaits; returns None; open-loop timing.

flip(direction)

direction: forward/backward/left/right; required. Auto-awaits; returns None; a missing acknowledgement raises.

set_axis(axis, power)

axis: pitch/roll/yaw/gaz/altitude; power clamps -100..100. Returns None immediately; persists until reset.

reset_motion()

No args. Returns None synchronously; zeros persistent axes and does not land.

emergency_cutoff()

No args. Auto-awaits; returns None. Immediate motor cutoff only; Bluetooth failure raises.

DIRECTIONS AND SAFETY

fly direction is up/down/left/right/forward/backward. flip uses forward/backward/left/right. Test short, low-power moves first; put land() in finally after take_off().

State, photos, and accessories

Return values can be stored in variables and tested in decisions. Photo capture saves the current camera frame to the session gallery.

battery_level()

No args. Returns number 0..100 or None before telemetry; synchronous and no expected failure.

is_flying()

No args. Returns bool from the controller's flying/landed state; synchronous.

is_landed()

No args. Returns bool from the controller's flying/landed state; synchronous.

wait_for_battery_change()

No args. Auto-awaits; returns None on a new value or Stop; transport loss may raise.

take_photo()

No args. Auto-awaits; returns None after requesting a photo; camera/transport failure raises.

open_grabber()

No args. Auto-awaits; returns None. Raises if no physical grabber is attached; simulator is a timing stub.

close_grabber()

No args. Auto-awaits; returns None. Raises if no physical grabber is attached; simulator is a timing stub.

fire_gun()

No args. Auto-awaits; returns None. Raises if no physical cannon is attached; simulator is a timing stub.

key_pressed(key)

key: string such as ArrowUp or a. Returns live bool synchronously; no expected failure.

stopped()

No args. Returns bool synchronously; True after runtime cancellation. It does not itself land.

LOW-BATTERY-CHECK.PY

```
1 battery = battery_level()
2 if battery is not None and battery < 25:
3     land()
```

Threshold vision and saved photos

Threshold and coverage use percentages from 0 to 100. Each sees or scan command captures a fresh frame.

`scan_threshold(threshold=60, invert=False)`

threshold: 0..100%; invert: bool. Auto-awaits; returns result record; missing camera raises.

`sees_binary(color, threshold=60, invert=False, coverage=10)`

color: white/black; coverage 0..100%. Auto-awaits a fresh frame; returns bool; camera failure raises.

`binary_center(color, threshold=60, invert=False)`

color: white/black; threshold 0..100%. Auto-awaits a fresh frame; returns bool; camera failure raises.

`take_photo()`

No args. Auto-awaits; returns None after saving/requesting the current view; camera/transport failure raises.

WHITE-PAPER-SEARCH.PY

```
1  take_off()
2  try:
3      for attempt in range(20):
4          if binary_center("white", 60, False):
5              take_photo()
6              break
7      fly("forward", 0.4, 12)
8  finally:
9      land()
```

FRESH FRAME + RESULT

Predicates scan before deciding. `scan_threshold()` returns threshold, invert, whiteCoverage, blackCoverage, centerWhite, frameWidth, frameHeight, and binaryData.

COCO objects and custom labels

Confidence is a fraction from 0 to 1 in Python. The built-in detector draws boxes; Teachable Machine classifies the whole frame.

`load_object_model()`

No args. Auto-awaits; returns model handle. Local-server/model-load failure raises.

`scan_objects(confidence=0.55)`

confidence: 0..1. Auto-awaits; returns up to 10 detection records; model/camera failure raises.

`detect_objects(confidence=0.55)`

Exact alias of `scan_objects()`. Same confidence, automatic wait, detection-list return, and failure behavior.

`sees_object(label, confidence=0.55)`

label: string; confidence 0..1. Auto-awaits a fresh scan; returns bool; model/camera failure raises.

`object_coordinate(label, axis, confidence=0.55)`

label string; axis x/y. Returns saved -100..100 number synchronously, or 0 when absent/below confidence.

`object_x(label, confidence=0.55)`

label: string; confidence 0..1. Synchronous x shortcut; returns saved -100..100 number or 0.

`object_y(label, confidence=0.55)`

label: string; confidence 0..1. Synchronous y shortcut; returns saved -100..100 number or 0.

OBJECT-DECISIONS.PY

```
1 if sees_object("person", confidence=0.45):
2     x = object_x("person", 0.45)
3     print(f"person x = {x}")
4
5 # Load model files in the UI first.
6 if sees_custom_label("red flag", 0.75):
7     take_photo()
```

`scan_custom_model()`

No args. Auto-awaits; returns class/probability records. Raises until model files are loaded or when camera fails.

`sees_custom_label(label, confidence=0.75)`

label: string; confidence 0..1. Auto-awaits; returns bool; unloaded model/camera failure raises.

AprilTags: detect, decide, and align

Hopper uses tag36h11 IDs 0 through 586. Use "any" when the specific ID does not matter.

`scan_april_tags()`

No args. Auto-awaits; returns detection records; missing camera/canvas raises.

`sees_april_tag(id="any")`

id: "any" or tag36h11 integer 0..586. Auto-awaits a fresh scan; returns bool; camera failure raises.

`sees_april_tag(id=42)`

Named-argument example for one ID. Same bool return, automatic waiting, and camera failure behavior.

`center_on_april_tag(id="any", power=10, center_slack=5, angle_slack=5, lost_searches=3)`

power 0..100; slacks 1..35/1..45; lost 1..20. Auto-awaits; bool; false on loss/Stop/30 s timeout.

TAG-MISSION.PY

```
1  take_off()
2  try:
3      if sees_april_tag(7):
4          centered = center_on_april_tag(7)
5          if centered:
6              take_photo()
7      else:
8          print("Tag 7 not found")
9  finally:
10     land()
```

DETECTION RETURN

Each record includes ID, center x/y on -100..100, image corners, and 2D image yaw. Centering commands roll/pitch/yaw only; they do not control height or land.

Variables, values, printing, and operators

A variable is created with `=`. Python uses `True`, `False`, and `None`; strings may use single or double quotes.

VALUES.PY

```
1 speed = 15
2 direction = "forward"
3 found = False
4
5 fly(direction, 1, speed)
6 found = sees_object("person")
7 print(f"Found: {found}")
```

COMPARISONS

`==` equal `!=` not equal `<` `<=` `>` `>=` `is` `None` `is not` `None`

`print(value)`

value: any; one or more arguments allowed. Returns `None` after writing to the app console; synchronous.

`len(value)`

value: string/list-like or `None`. Returns integer length (0 for `None`); synchronous.

`contains(collection, value)`

collection: string/list/set/map-like; value: any. Returns bool synchronously; unsupported collections return `False`.

`abs(value)`, `min(value1, value2)`, `max(value1, value2)`, `round(value)`

Numeric arguments. Return numbers synchronously using the browser math helpers; invalid values may produce `NaN`.

`int(value)`, `float(value)`, `str(value)`, `bool(value)`

value: any. Return converted value synchronously; `int` truncates; conversion follows JavaScript coercion.

BOOLEAN LOGIC

`and` requires both; `or` requires either; `not` reverses `True/False`. Parentheses make combined decisions easier to read.

Make decisions with if, elif, and else

Only the first matching branch runs. Every nested line is indented four spaces.

BATTERY-DECISION.PY

```
1  battery = battery_level()
2
3  if battery is None:
4      print("Waiting for battery data")
5  elif battery < 25:
6      print("Battery too low")
7      land()
8  else:
9      take_off()
```

VISION-DECISION.PY

```
1  if sees_object("person", 0.5):
2      hover()
3  else:
4      fly("forward", 0.4, 12)
```

COMBINE-TESTS.PY

```
1  if is_flying() and not stopped():
2      take_photo()
```

COMMON ERROR

A missing colon or inconsistent indentation stops translation before the flight run begins. The console points to the Python line.

Repeat with for and safe while loops

Loops yield control between passes so the red Stop & Land button remains responsive.

FOR-LOOP.PY

```
1 # Repeat exactly four times.
2 for step in range(4):
3     fly("forward", 0.5, 12)
4     rotate(90)
```

range(stop)

stop: finite number. Returns list from 0 to before stop with step 1; range(4) -> 0,1,2,3.

range(start, stop)

start/stop: finite numbers. Returns list from start to before stop; range(2, 6) -> 2,3,4,5.

range(start, stop, step)

step: finite nonzero number. Returns ascending/descending list; invalid numbers or zero step raise.

WHILE-LOOP.PY

```
1 # Repeat until found or stopped.
2 while not stopped():
3     if sees_object("stop sign"):
4         hover()
5         break
6     fly("forward", 0.4, 10)
```

LOOP CONTROLS

break leaves the nearest loop. continue skips to the next pass. pass is a temporary empty line. Prefer while not stopped(): for open-ended flight searches.

Functions and guaranteed landing

Use def to name a reusable sequence. Hopper automatically waits when a student-defined function is called.

FUNCTIONS.PY

```
1 def search_leg(seconds, power):
2     fly("forward", seconds, power)
3     hover()
4     return sees_object("person")
5
6 take_off()
7 try:
8     found = search_leg(1, 12)
9     print(f"Found: {found}")
10 finally:
11     land()
```

DEF

Function parameters are simple names. return sends a value back. Calls may appear in an assignment or decision.

TRY / FINALLY

The finally section runs after success or error. Put land() there when a mission could fail after takeoff.

EXCEPT

Use except Exception as error: to log a problem with print(error), then land in finally.

Quick reference: units, rules, and support

Keep this page nearby during a lab. Use the page map to jump to each fully documented command group.

UNITS

Power and threshold: 0-100 percent Confidence: 0-1 fraction Time: seconds Rotation: degrees Vision x/y: -100 to +100

FLIGHT + STATE API

Pages 3-4 document lifecycle, motion, timing, state, photos, accessories, keys, cancellation, argument types/defaults, returns, automatic waiting, and failures.

FIND A COMMAND FAST

Flight + timing: p3 State + accessories: p4 Threshold: p5 Objects + custom: p6 AprilTags: p7 Values + logic: p8-9 Loops + functions: p10-11

DIRECTIONS

fly: up, down, left, right, forward, backward flip: forward, backward, left, right rotate: clockwise, counterclockwise

VISION API

Pages 5-7 document thresholding, COCO objects, custom labels, AprilTags, units, defaults, return records, automatic waiting, and camera/model failures.

PYTHON RULES

Four spaces per level. Colon after block headers. Calls stay on one line. No await or import. Use True, False, None, and snake_case command names.

LANGUAGE TOOLS

Pages 8-11 cover values, printing, operators, decisions, bounded loops, functions, return values, and guaranteed landing.

SAFETY

Use conservative power and short motion times. Test vision while landed. Keep the area clear. Stop & Land remains the operator control; emergency_cutoff is for immediate motor shutdown only.

Source of truth: [lib/python.ts](#), [lib/drone.ts](#), [lib/vision.ts](#), and [lib/runtime.ts](#)